Լաբորատոր աշխատանք 8

Աշխատանքի նպատակ

Ստեղծել `SolarStation` դաս՝ արևային կայանի արտադրանքը հաշվարկելու համար՝ կիրառելով կոնստրուկտորների գերբեռնումը։

Խնդրի պահանջ

Ստեղծել `private` դաշտեր և երեք `public` կոնստրուկտոր՝ առանց պարամետրերի, երեք և չորս պարամետրերով։ Իրականացնել տվյալների ստուգում, փոփոխող մեթոդ, համախառն և օգտակար արտադրանքի հաշվարկ։

Առաջադրանք

Ստեղծել երեք օբյեկտ՝ տարբեր կոնստրուկտորներով, ապա 5 կայանից բաղկացած զանգված։ Գտնել ամենամեծ օգտակար արտադրանքով կայանը և հաշվարկել օգտակար արտադրանքների գումարը։

C++ կոդ

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

class SolarStation {
private:
    string name;
    int days;
    double dailyProduction, losses;

public:
    // Default constructor
    SolarStation() {
        name = "Unknown";
```

```
    days = 1;
    dailyProduction = 1;
    losses = 0;
}

// Three-parameter constructor
SolarStation(string n, int d, double p) {
    name = n.empty() ? "Unknown" : n;
    days = d > 0 ? d : 1;
    dailyProduction = p > 0 ? p : 1;
    losses = 0;

    if (d <= 0 || p <= 0)
        cout << "Invalid value. Default used.\n";
}

// Four-parameter constructor
SolarStation(string n, int d, double p, double l) {
    name = n.empty() ? "Unknown" : n;
    days = d > 0 ? d : 1;
    dailyProduction = p > 0 ? p : 1;
    losses = (l >= 0 && l <= 25) ? l : 0;

    if (d <= 0 || p <= 0 || l < 0 || l > 25)
        cout << "Invalid value. Default used.\n";
}

// Add working days
void addDays(int d) {
    if (d > 0)
        days += d;
    else
        cout << "Days must be positive.\n";
}

// Gross production
double grossProduction() const {
    return days * dailyProduction;
}
```

```
    // Useful production
    double usefulProduction() const {
        return grossProduction() * (1 - losses / 100);
    }

    // Display station data and calculations
    void show() const {
        cout << "Name: " << name
             << "\nDays: " << days
             << "\nDaily production: " << dailyProduction
             << "\nLosses: " << losses << "%"
             << "\nGross production: " << grossProduction()
             << " kWh\nUseful production: "
             << usefulProduction() << " kWh\n\n";
    }
};

int main() {
    cout << fixed << setprecision(2);

    // Three objects using different constructors
    SolarStation s1;
    SolarStation s2("Station A", 10, 40);
    SolarStation s3("Station B", 20, 50, 10);

    cout << "Three objects:\n";
    s1.show();
    s2.show();
    s3.show();

    // Array of five stations
    SolarStation stations[5] = {
        SolarStation("Station 1", 10, 40, 5),
        SolarStation("Station 2", 20, 50, 10),
        SolarStation("Station 3", 15, 60, 8),
        SolarStation("Station 4", 12, 45, 15),
        SolarStation("Station 5", 25, 30, 5)
    };
```

```
// Input data for five stations
for (int i = 0; i < 5; i++) {
    string name;
    int days;
    double production, losses;

    cout << "Enter name, days, daily production, losses: ";
    cin >> name >> days >> production >> losses;

    stations[i] = SolarStation(name, days, production, losses);
}

// Modify one station
stations[0].addDays(5);

// Find station with maximum useful production
int maxIndex = 0;
double total = 0;

for (int i = 0; i < 5; i++) {
    total += stations[i].usefulProduction();

    if (stations[i].usefulProduction() >
        stations[maxIndex].usefulProduction()) {
        maxIndex = i;
    }
}

cout << "\nStation with maximum useful production:\n";
stations[maxIndex].show();

cout << "Total useful production: "
     << total << " kWh\n";

// Verification example: 20 * 50 = 1000, useful = 900
SolarStation test("Test", 20, 50, 10);
cout << "\nVerification example:\n";
test.show();
```

```
    return 0;
}
```